

Adaptive Resource Efficient Microservice Deployment in Cloud-Edge Continuum

Kaihua Fu, *Student Member, IEEE*, Wei Zhang^{ID}, *Student Member, IEEE*, Quan Chen^{ID}, *Member, IEEE*, Deze Zeng^{ID}, *Member, IEEE*, and Minyi Guo^{ID}, *Fellow, IEEE*

Abstract—User-facing services are now evolving towards the microservice architecture where a service is built by connecting multiple microservice stages. Since the entire service is heavy, the microservice architecture shows the opportunity to only offload some microservice stages to the edge devices that are close to the end users. However, emerging techniques often result in the violation of Quality-of-Service (QoS) of microservice-based services in cloud-edge continuum, as they do not consider the communication overhead or the resource contention between microservices and external co-located tasks. We propose Nautilus, a runtime system that effectively deploys microservice-based user-facing services in cloud-edge continuum. Nautilus ensures the QoS of microservice-based user-facing services while minimizing the required computational resources, which is comprised of a communication-aware microservice mapper, a contention-aware resource manager and an IO-sensitive and load-aware microservice migration scheduler. The mapper divides the microservice graph into multiple partitions based on the communication overhead and maps the partitions to appropriate nodes. On each node, the resource manager determines the optimal resource allocation for its microservices based on reinforcement learning that may capture the complex contention behaviors. Once the microservices are suffered from external IO pressure, the IO-sensitive microservice scheduler migrates the critical one to idle nodes. Furthermore, when the load of microservices changes dynamically, the load-aware microservice scheduler migrates microservices from busy nodes to idle ones to ensure the QoS goal of the entire service. Our experimental results show that Nautilus can guarantee the required QoS target under external shared resources contention while the state-of-the-art suffers from QoS violations. Meanwhile, Nautilus reduces the computational resource usage by 23.9% and the network bandwidth usage by 53.4%, while achieving the required 99%-ile latency.

Index Terms—Cloud-edge continuum, QoS, microservice resources management

1 INTRODUCTION

USER-FACING applications often have strict Quality-of-Service (QoS) requirements in terms of tail latency, and usually require frequent bug fixes as well as feature updates. To this end, the design of user-facing services is shifting from a monolithic architecture to a microservice architecture [1], where a complex user-facing service is decomposed into multiple loosely coupled microservice stages. A microservice-based application involves the interoperation of multiple microservices, each of which can be implemented and updated independently. Such independence improves the application's scalability, portability, and availability. Considering these advantages, the microservice architecture has been regarded as the widely accepted and employed software

architecture by Internet giants such as Netflix, Amazon, and Apple [2], [3], [4].

A traditional monolithic user-facing application is often entirely deployed in the datacenter [5], [6], [7], [8], as edge nodes are not powerful enough to host an entire application. The microservice architecture enables the opportunities to deploy some of an application's microservice stages on edge nodes that are close to end users. Deploying microservices in the cloud-edge continuum benefits from both the low access latency of edge nodes and the high computational ability of clouds. Internet companies are also exploring to deploy their services in the cloud-edge continuum [9], [10]. Moreover, for cost efficiency consideration, it is desirable to minimize the resource usage of a microservice-based user-facing application in the cloud-edge continuum, while still ensuring its QoS requirement.

Prior works on the microservice deployment in the cloud only consider the computational and memory resources [11], [12], [13]. On the contrary, as the edge nodes are usually geographically distributed, the edge-edge and edge-cloud communications go through unstable public networks. Fig. 1 describes the microservice deployment in a cloud-edge continuum. As shown in Fig. 1, the computational ability of each node, the resource contention on each node, and the edge-to-edge and edge-to-cloud communication overhead together affects the end-to-end latency of a user-facing application. Therefore, prior works on microservice deployment in cloud datacenters do not apply to the cloud-edge continuum.

- Kaihua Fu, Wei Zhang, Quan Chen, and Minyi Guo are with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China. E-mail: {midway, zheng-w}@sjtu.edu.cn, {chen-quan, guo-my}@cs.sjtu.edu.cn.
- Deze Zeng is with the School of Computer Science, China University of Geosciences, Wuhan, Hubei 430074, China. E-mail: deze@cug.edu.cn.

Manuscript received 24 May 2021; revised 14 Sept. 2021; accepted 10 Nov. 2021. Date of publication 15 Nov. 2021; date of current version 9 Dec. 2021.

This work was supported in part by the National R&D Program of China under Grant 2018YFB1004800, the National Natural Science Foundation of China (NSFC) under Grants 62022057, 61832006, 61632017, and 61872240, the Open Research Projects of Zhejiang Lab under Grant 2021KE0AB02. (Corresponding author: Quan Chen.)

Recommended for acceptance by B. Di Martino.

Digital Object Identifier no. 10.1109/TPDS.2021.3128037

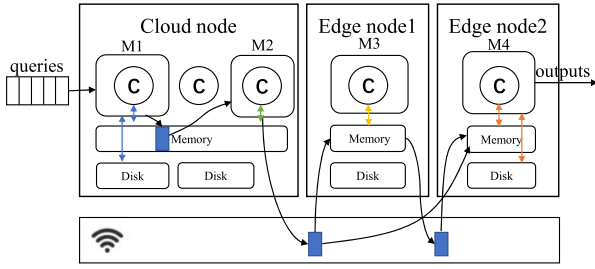


Fig. 1. Deploying a user-facing application with four microservice stages in cloud-edge continuum with one cloud node and two edge nodes.

Besides, the microservice applications often consume more computational resources (e.g., CPU cores and memory capacity) and network bandwidth while they consume low-level disk IO bandwidth based on our insight. Hence it's potential to improve the resource utilization of nodes by co-locating IO-intensive tasks with microservices on the same node. However, this kind of co-location brings extra challenges due to the inevitable shared resource contention, which results in the performance degradation of microservices in the node. Specifically, the co-located tasks will compete for IO shared resources with the load variation, which will cause external pressure for microservices.

There have been some orchestration systems that can be used to deploy microservices in the cloud-edge continuum, such as Kubernetes [14] and Rhythm [11]. On one hand, Kubernetes randomly schedules the microservices to the nodes while users are able to give higher privileges to some critical microservices through allocating computational resources on each node. However, it lacks the ability to identify the critical microservices that determine the QoS of an application with a complex dependency relationship between microservice stages. With the dynamic load changes of microservice applications and the external IO pressure suffered from the co-located IO-intensive tasks, the critical microservices changes accordingly. On the other hand, Rhythm maps microservices to a distributed cloud environment based on microservices' functions and allocates computational resources for microservices on demand. However, it does not consider the contention between the microservices on the same node, resulting in QoS violations when resource-constrained edge servers are involved.

Considering the above limitations of existing microservice orchestration strategies, four main challenges have to be resolved to achieve the above purpose that ensures the QoS of a microservice-based user-facing application while minimizing the used resources in the cloud-edge continuum. Challenge 1: the communication through the public network may result in long latency. A method is required to effectively map the microservice stages to the cloud nodes and edge nodes while minimizing the communication overhead. Challenge 2: the shared resource contention between microservices on the same node result in QoS violation. Allocating more resources to a microservice may slow down other microservices on the same node. Challenge 3: co-locating IO-intensive tasks with microservices may affect the microservice performance. When the IO shared resources of microservices are occupied by external tasks, it is difficult to allocate more resources for microservices on a single node. We consider migrating microservices from busy nodes, however, a random migration

cannot effectively improve the quality of service of the overall microservice application. Challenge 4: the optimal microservice deployment shall vary accordingly to the load of a user-facing application since microservices compete for computational resources with each other and the QoS of an application is affected by almost all its constituent microservices. For instance, a microservice on an edge node has to be migrated to the cloud, when the load increases and the edge node fails to provide the required computational resources.

Since contention for shared resources (challenge 2 and 3) and load changes of microservices (challenge 4) can only be obtained at runtime, it's unwise to design a static strategy. To this end, we propose *Nautilus*, a runtime system that is comprised of a *communication-aware microservice mapper*, a *contention-aware resource manager*, an *IO-sensitive microservice scheduler*, and a *load-aware microservice scheduler*. For a microservice-based application to be deployed, the data dependency between the microservices connects the microservices to be a graph. The microservice mapper partitions the microservice graph of services into multiple parts and maps them to the edge nodes and cloud nodes. The partition and the initial mapping are determined based on the communication overhead between the microservices. For the microservices on each node, the resource manager determines the optimal resource allocation (cores, memory, and network bandwidth) for each microservice using reinforcement learning (RL). RL is used to capture the complex interactive relationship between the performance of the microservices due to the contention.

Meanwhile, *Nautilus* monitors the tail latency of overall microservice application, load pressure, and resource utilization pressure of each node, and if the current mapping tends to suffer from QoS violation, *Nautilus* migrates suitable microservice from busy node to other idle nodes to guarantee the quality of service. If microservices are under the external IO pressure from co-located IO-intensive tasks which causes the load imbalance, the IO-sensitive scheduler adaptively identifies and migrates the disturbed bottleneck microservices. In addition, when the pressure of the computational resources between the microservices increases, the load-aware scheduler will select the most suitable microservices for migration to ensure the quality of service while minimizing the change in communication volume caused by the migration.

In summary, the microservice mapper and the two schedulers determine the mapping of the microservices to the nodes, and the resource manager determines to amount of resources allocated to each microservice. This paper makes four main contributions.

- *Communication overhead aware microservice mapping.* We propose a graph-based microservice mapper to make frequent data interaction complete in memory, thereby reducing communication overhead.
- *Resource contention aware resource allocation.* We introduce a resource manager based on a RL algorithm to capture the resource contention to the overall performance, and accordingly make optimal resource allocation to lower the resource usage.
- *IO-sensitive microservice migration.* We design an IO-sensitive critical microservice scheduler to migrate

critical microservices affected by external co-located IO-sensitive tasks to guarantee the required QoS requirements of microservices.

- *Load dynamics sensitive microservice migration.* A runtime load-aware scheduler migrates the microservices from busy nodes to idle ones to reduce resource contention in response to the load dynamics while minimizing the communication overhead.

We evaluate Nautilus on a cloud-edge continuum built with Alibaba Cloud, personal laptop, and Raspberry Pi 4B+. The experimental results show that Nautilus will guarantee the required 99%-ile latency under external IO pressure while the state-of-the-art will suffer from QoS violations. Meanwhile, Nautilus will reduce the use of computing resources and network resources by 23.9% and 53.4%, respectively, while achieving the required 99%-ile latency.

The rest of the paper is organized as follows: Section 2 introduces the representative prior work. In Section 3, we investigate the background and the challenges of deploying microservices in the cloud-side continuum. Section 4 introduces the design ideas of Nautilus. Section 5 presents how we map microservices to adapted computing nodes. In Section 6 we propose a contention-aware resource manager to allocate shared resources to each microservice. In Sections 7 and 8, we respectively introduce two migration strategies to eliminate runtime QoS violations. We evaluate the effectiveness of Nautilus in Sections 9 and Section 10. Finally, we conclude this paper in Section 11.

2 RELATED WORKS

There have been some prior work on related topics, such as resources task placement and resources management in cloud and edge, and microservice-based architectures.

Resources Management in Cloud and Edge. Prior work [15], [16], [17], [18], [19], [20], [21], [22] allocate the computational resources to the co-located applications for maximizing the throughput of batch applications while guaranteeing the required QoS of user-facing services on cloud datacenter. CODA [23] eliminates the resource contention of multi-stage DNN training tasks to improve the resource utilization of the datacenter. However, CODA is oriented to batch processing tasks and is not suitable for latency-sensitive Internet services. CLITE [8] and Twig [24] co-locate multi-latency web services with batch applications to maximize the resources utilization while avoiding the QoS violations for services on datacenters. PowerChief [25] designs key component acceleration strategies to ensure the low latency target of multi-stage applications. However, these works only consider CPU, LLC, and memory bandwidth contention, since they target Internet-oriented services that do not perform IO operations and network communications frequently.

Some state-of-the-arts extensions analyze more resource interference, they treat disks as shared resources and allocate IO bandwidth for multi-tenant tasks co-located on the same node. Kassiano proposes an IO contention-aware resource management framework [26] for big data tasks, but the framework is not suitable for latency-sensitive Internet services. DIAL and Quasar [27], [28] analyze the resource categories that interfere with workloads and deploy workloads that do not affect each other's performance in a cluster to

improve throughput. PARTIES and Ursa [6], [29] analyze the resource consumption of co-located applications and allocate "just enough" shared resources to the applications to reduce resource usage and improve fairness without hurting the performance of the applications. These co-location methods ignore the dependency relationship between microservices, which are not applicable to microservice-based applications. Rhythm [11] aggregates microservice-based applications that are sensitive to different resources into the same node, which improves the anti-interference ability of services and achieving higher throughput. In addition, some other works have focused on task placement and resource management in cloud-edge continuums [30], [31], [32], [33]. However, these works did not consider the resource contention of microservices at runtime.

Microservice Architecture. Peng [34] *et al.* and Gan [1] *et al.* focus on building benchmark suites based on microservice architecture. They use benchmarks to analyze the characteristics of microservices and the implications in networks and operating systems. Atom [35] and Hyscale [36] combine horizontal and vertical scaling to determine the resource partition of each microservice stage to improve the performance of microservices. Wisp [37] and DAGOR [38] have focused on load balance on microservice architecture and designed microservice load adjustment mechanisms by setting different priorities. Seer[13], Sinan[39], and Sage [40] use ML-based methods to predict whether microservice applications are subject to QoS violations and locate the critical stages that lead to QoS violations. Carrusca *et al.* [41] designs an adaptive migration mechanism for microservices in the cloud environment to reduce the response delay of services. Filip *et al.* [42] studies the cloud-edge microservices deployment model based on heterogeneous platforms and simulated it on the simulator. Hou *et al.* [12], [43] are committed to deploying microservices in data centers and reducing power consumption while guaranteeing the required QoS target. Amoeba [44] dynamically switches the deployment mode of microservices in IaaS and serverless computing to reduce the usage of shared resources of microservices. Dagger[45] proposes an FPGA-based RPC hardware acceleration structure to improve the communication efficiency between microservices in the same node. There are a lot of other works that have contributed to the performance improvement of microservices, such as [46]. However, due to the lack of consideration of communication overhead between distributed nodes, these researches may lead to end-to-end latency violations.

3 BACKGROUND AND MOTIVATIONS

In this section, we investigate the effectiveness of emerging frameworks for scheduling microservice-based user-facing applications in cloud-edge continuum.

We use all the 3 open-sourced benchmarks *SocialNetwork* (SN), *MediaService* (MS), and *HotelReservation* (HR) in the DeathStarBench [1] microservice benchmark suite, and a microservice-based train ticket booking system *TrainTicket* (TT)[34] to perform the investigation. These benchmarks show different characteristics and thus cover a large spectrum of real-system applications.

We build a real-system cloud-edge continuum to run the experiment in the following of this paper. The experimental

TABLE 1
Hardware and Software Setup

Specification	
Hardware	Cloud: Intel Xeon Platinum 8269CY CPU @ 2.50GHz 8GB Memory Capacity
	Desktop: Intel Core i7-9700CPU @ 3.00GHz 6GB Memory Capacity Raspberry Pi 4b+
Software	Ubuntu 18.04.4 LTS with kernel 5.3.0-40-generic Docker version 19.03.6 Kubernetes version 1.18.0
Benchmark	SocialNetwork (SN), MediaService (MS), HotelReservation (HR), Train-Ticketing (TT)

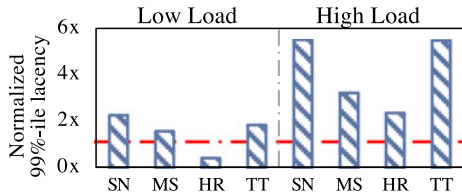


Fig. 2. The 99%-ile latencies of the benchmarks with Kubernetes.

cloud-edge platform is built with a cloud node in Alibaba Cloud Elastic Server [47], a Raspberry Pi 4B+, and a personal desktop as the edge nodes. The detailed hardware setup is summarized in Table 1. As to the external IO pressure, we use *fiio* [48] to co-locate with microservices in the real environment to increase the throughput of a single node.

3.1 QoS Violation and Resource Inefficiency

By deploying microservices of a user-facing application in the cloud-edge continuum, it is expected to explore the advantages of the high cloud computational ability and the low edge access latency. Kubernetes [14] is one of the most popular microservice scheduling frameworks. Adopting the best-effort strategy, Kubernetes randomly maps each microservice to the nodes in the cloud-edge continuum. On each node, the resources are freely shared by the microservices.

Fig. 2 shows the normalized 99%-ile latencies of the benchmarks with Kubernetes in the cloud-edge continuum. As shown in the figure, 3 out of the 4 microservice benchmarks suffer from QoS violation at low load, and all of them suffer from QoS violation at high load. The QoS violation originates from the poor scheduling of Kubernetes, as it does not consider the communication overhead between the microservices, and the actual resource contention between the microservices on the same node (to be explained in the next subsection).

As an example, Fig. 3 shows the actual CPU usage increment of the microservices in the benchmark SN at the high load compared with Nautilus. As observed from the figure, 11 out of the total 12 microservice stages occupy excessive resources, even though SN still suffers from the QoS violation. On the contrary, our study shows that it is possible to ensure QoS with fewer CPU cores (Fig. 17). The resource inefficiency originates from the poor resource allocation of Kubernetes. As Kubernetes treats all the microservices equally, it may allocate an excessive amount of shared computational resources to the microservices that are not

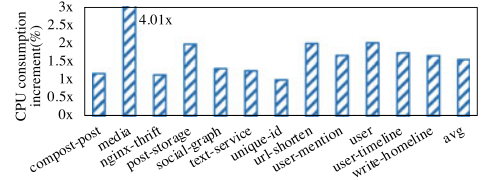


Fig. 3. The CPU resource allocated to each microservice with Kubernetes. If *y*-axis is larger than one, the microservice is allocated more CPU resource than its actual need.

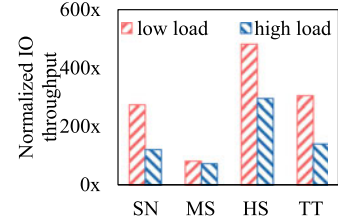


Fig. 4. The IO resources usage of the node when microservices are co-located with IO-sensitive task.

on the latency-critical path, but insufficient resources to the critical microservices.

In summary, existing cloud-oriented microservice scheduling frameworks do not fit for on cloud-edge continuum as they may result in QoS violation and poor shared resource utilization efficiency.

3.2 Necessity for IO-Sensitive Microservice Migration

Based on our insight, we find that the IO bandwidth resources consumed by the 4 microservices are at a low level, usually less than 2 MB/s IO bandwidth consumption, which is not obvious to the pressure generated by disk shared resources. So it's potential to co-locate some IO-intensive tasks with microservice applications on the same node to improve the utilization of IO resources.

We co-locate 4 microservice applications with IO-intensive task *fiio* in the same cluster. Fig. 4 shows the increase in throughput of disk bandwidth under colocation. We define the normalized IO throughput as the ratio of disk bandwidth consumption to microservice application disk bandwidth usage. As observed from Fig. 4, the utilization of disk bandwidth has increased by an average of 220 times in 8 scenarios (4 microservice applications under low load and high load) which greatly increases the throughput of the IO bandwidth.

However, the runtime performance of microservices receives contention for shared resources when co-located with IO-intensive tasks, which leads to an increase in end-to-end latency. To further observe the contention of shared resources, As for an example, Fig. 5 shows the performance degradation of each IO sensitive microservice in *MediaService* when colocating with IO sensitive tasks in high load. *Base* is the duration of a microservice when it is running alone, and *co-locate* represents the performance of the microservice under real IO pressure.

It can be observed from Fig. 5 that the actual duration of all microservices under IO pressure is higher than their stand-alone duration. This is mainly because there is the shared resource contention (e.g., pagecache) between microservices

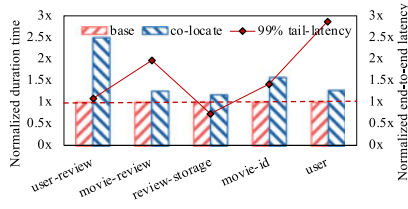


Fig. 5. The duration of microservice under IO resources contention. The red line shows the normalized 99%-ile latencies after migrate each microservice to node without IO contention.

and co-located IO tasks [49], which makes performance imbalance. In severe cases, microservices will suffer 2.4 times the performance loss. Therefore, it is necessary to migrate IO-sensitive microservices to other idle nodes when there is external IO pressure. However, unreasonable migration cannot significantly improve the quality of service. For example, we once map one of the microservices to idle nodes without co-located tasks, while others suffer from IO pressure. Fig. 5 shows the end-to-end latency under five situations (migrating five different microservices). Observe from the figure, only when the microservice *review-storage* is migrated, the end-to-end latency can be guaranteed. Due to the complex topology of microservice applications, the diversity of duration of microservices does not directly affect the overall end-to-end latency. On the contrary, the QoS of the microservice application can only be guaranteed when the duration of the critical microservice is not disturbed. Therefore, how to identify which critical microservice can be migrated becomes indispensable.

Besides, Kubernetes may schedule the microservices that contend for shared computational resources (such as CPU core and memory capacity, etc) seriously to the same node. We omit the results on the performance interference between the co-located microservices, which has been extensively proved [13]. In this case, the performance of a microservice may be severely affected by other microservices on the node with limited resources, thus resulting in overall performance degradation.

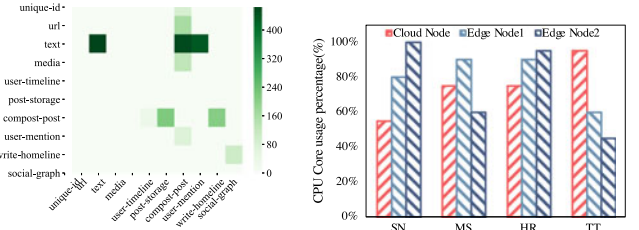
3.3 Diving into the Microservice Deployment

It has been observed that the “best effort” deployment strategy leads to serious QoS violations in Section 3.1. To understand the problems of QoS violation and resource inefficiency, we dive into the details of the microservice deployment in the cloud-edge continuum.

As shown in Fig. 1, the microservices on different nodes communicate through the public network, and the microservices on the same node contend for the shared resources. Therefore, the communication overhead and the performance of the microservices together affect the QoS of a microservice-based application.

As already observed from Fig. 2, Kubernetes results in the QoS violation. This is mainly because it does not consider the *communication overhead*, or the *balance of resource usage* when scheduling and allocating shared resources for microservice stages.

To further understand the reasons behind, Fig. 6a shows the amount of data transferred between microservices in the benchmark SN. As shown in the figure, the microservice *text* transfers a large volume of data to the microservices



(a) The size of data transferred (b) CPU usage of each node in high load

Fig. 6. The problem of Kubernetes deployment. The microservice applications suffer from high communication overhead and imbalanced load.

compost-post and *user-mention*. The data transferred between the other microservices are in small size. Intuitively, it is better to map the microservices *text*, *compost-post*, and *user-mention* on the same node if the node has enough computational resources (e.g., the node in the cloud). However, we observe that Kubernetes maps the three microservices on three nodes. In this case, the microservice *text* has to transfer a large amount of data to *compost-post* and *user-mention*. The long data transfer time results in a long response time. Kubernetes makes poor microservice mapping decisions because it does not consider the communication overhead between microservices.

Furthermore, Kubernetes does not adapt to the load variation, which is inherent in user-facing services. When the load of a user-facing application is low, all its microservices only require a small amount of computational resources (cores, memory capacity), and most of the nodes are able to satisfy the resource requirements. When the load increases, the resource requirements of different microservices also increase, but with different extents. Such diversity may cause resource shortages on some nodes. Fig. 6b reports the resource usage of each node at high load. As shown in the figure, for SN, the CPU usage of “Edge Node2” is much higher than that of the cloud node, potentially leading to QoS violation. In this case, if some microservices can be migrated from busy nodes to idle nodes on demand, the QoS violation problem can be alleviated.

3.4 Challenges of Nautilus

We therefore design and implement *Nautilus* to minimize the resource usage while ensuring the QoS of microservice-based user-facing applications. Based on the above analysis, *Nautilus* should consider the communication overhead, the contention between microservices, and the resource usage balance. Specifically, *Nautilus* has to resolve four key challenges before achieving the above purpose.

- *Communication between microservices in different nodes is prone to high communication overhead.* It is necessary to consider the microservice topology to deploy closely related microservices to the same node to ease the communication overhead.
- *Shared resource management of microservices must be based on runtime information.* The resource contention of co-located microservices and the complex structure of microservices makes it difficult to determine the critical parts of microservices through static information.

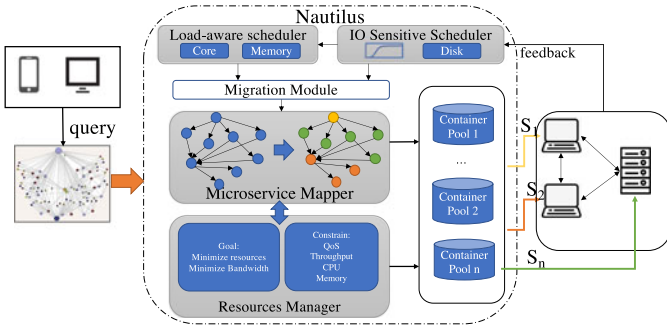


Fig. 7. The design overview of Nautilus.

- The shared resources contention from colocated IO-sensitive tasks has caused QoS violations in microservice applications. It is expected to design an adaptive mechanism to identify bottleneck microservices and eliminate the impact of IO contention on the overall end-to-end latency.
- The variable microservice load pressure leads to unbalanced resource partition. As the load pressure increases, the inefficiency of unbalanced shared resource partitions gradually manifests. It is desirable to design a strategy to balance the shared resource allocation to fully explore the advantages of the cloud-edge continuum.

4 METHODOLOGY OF NAUTILUS

In this section, we describe the design methodology of Nautilus. Based on the above analysis, we design Nautilus following four guidelines. 1) Nautilus should minimize the communication overhead between microservices. Since the data transfer between microservices suffers from high latency and low bandwidth. 2) Nautilus should minimize the microservice shared computational resources usage while guaranteeing QoS without any prior knowledge. 3) Nautilus should accurately identify the disturbed bottleneck IO-sensitive microservices and schedule them to the idle node. 4) Nautilus should balance shared resource consumption between nodes to improve the resource utilization of the cloud-edge continuum.

Fig. 7 shows the design overview of Nautilus. Nautilus is comprised of a *communication-aware microservice mapper*, a *contention-aware resource manager*, an *IO-Sensitive microservice scheduler*, a *load-aware microservice scheduler*. The mapper minimizes the communication overhead in the cloud-edge continuum. The resource manager minimizes the usage of the computational resources of microservice on the cloud-edge continuum at the low load and extends the supported peak load with limited resources when the load is high while ensuring the 99%-ile latency target. The IO-sensitive microservice scheduler accurately identifies the affected bottleneck IO-sensitive microservices at runtime and migrates them to idle nodes. The load-aware scheduler identifies QoS violations caused by load changes at runtime, and balance the resource usage of nodes. The migration module is integrated into the runtime system to execute the migration actions determined by the two schedulers.

Microservices mapper decides which microservice to place on which server (Section 5). The mapper places tasks with large data interactions on the same server so that the

data transformation of these services is completed through global memory, thereby reducing the communication delay.

On each node, the resource manager allocates computational resources (i.e., cores, memory capacity, network bandwidth) to microservices according to an online reinforcement learning algorithm and runtime contention behaviors (Section 6). The challenging part of the manager is decreasing performance degradation which is caused by resource contention between microservices.

Nautilus collects the probability distribution law of the duration of the IO sensitive microservice and establishes a criticality model to locate the critical IO microservices that lead to QoS violations (Section 7). The IO-sensitive scheduler migrates critical microservice stages to other idle nodes to eliminate the shared resource interference caused by external IO-intensive tasks to microservice applications.

When the load increases, the computational resource consumption of microservices increases sharply, causing microservices to compete to shared resources with each other, which may lead to unbalanced resource partition. Nautilus identifies busy nodes and migrates the microservices of the corresponding nodes to other idle nodes (Section 8).

We implement Nautilus by modifying Kubernetes [14]. All microservice-based applications for Kubernetes work in Nautilus without any application modification. Nautilus determines the location and resource consumption of each microservice. As can be seen in Fig. 7, the feedbacks $S_1, S_2, \dots, S_n$ represent the target node of each microservice and the computational resource quota, including CPU cores, memory capacity, and Network IO bandwidth. We regularly monitor the operation of microservices, including end-to-end latency, throughput, the execution time of each microservice, and resource quota occupation of each microservice, which should feedback to the corresponding component of Nautilus. In the next time interval, we adjust microservices computational resource usage and determine whether to migrate microservices based on feedback performance.

The migration module is implemented by Kubernetes *scale* instruction. When the load-aware scheduler and IO-sensitive scheduler determine to migrate the microservice, the migration module migrates the microservice from the source node to the target node. Specifically, we scale out the microservice instance to be migrated to the target node. After the instance is started on the target node, we delete the corresponding instance on the source node.

Specifically, the relationship between each module can be summarized as the following steps. 1) Microservice mapper will map each part of the microservice to one of the nodes according to the data transfer byte to minimize the communication overhead and use it as the initial state of the mapping. 2) Once all stages in the microservice are deployed on the nodes, the resource manager allocates computational resources (core, memory space, network bandwidth) for each microservice stage to avoid QoS violations while minimizing the usage of the computational resources. 3) If there are QoS violations at runtime, the initial mapping becomes inapplicable. Nautilus migrates microservices from congested nodes to other idle nodes. For detail, when the resource manager completes the allocation in each time interval, the IO-sensitive scheduler and load-aware scheduler determine whether to migrate microservices in turn. If

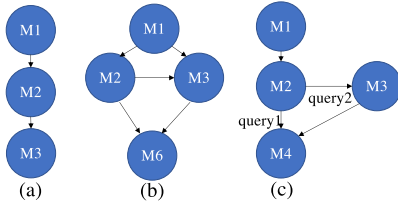


Fig. 8. Different types of structure used in microservice applications.

so, the migration module executes the migration actions of each scheduler in order. Meanwhile, Nautilus suspends online resource allocation until the microservice is migrated to the target node. There are two migration strategies: 4) The IO-sensitive scheduler records the duration of the IO-sensitive microservice as the IO latency. When the latency of IO-intensive microservices is detected to increase, the IO-sensitive microservice scheduler will transfer the critical IO-intensive microservices to idle nodes. 5) The load-aware scheduler records the sum of microservice resource quotas on each node. When the node has no free resource quota, the load-aware scheduler defines the node to be a congested node and migrates the microservice to an idle node.

5 COMMUNICATION-AWARE MICROSERVICE MAPPING

In this section, we present a Microservice mapper to solve the mapping problem between microservices and nodes, aiming at reducing the overall communication overhead between microservices.

5.1 Service-Node Mapping Problem

We find that if microservices are deployed on the same node, the data interaction between microservices can be transmitted through global memory instead of the network, potentially easing the pressure of network communication. Following such concepts, Nautilus maps the microservices with frequent data interactions onto the same node. However, as a large number of microservices compose a complex topological graph, determining which nodes to place microservices for minimizing the overall communication overhead is important.

The microservice structure can be described as a directed acyclic graph (DAG)[50] as shown in Fig. 8. Suppose Nautilus deploys n microservices in k nodes, we construct a DAG $G = (V, E)$, where vertices V are microservice and edges E refer to the communication between microservice, and the communication overhead between microservice are considered as the weight of the edge. After removing a subset of the edges $E' \subseteq E$, the topological graph of the microservice can be split into k independent subgraphs, and thereby the microservices corresponding to each subgraph can be deployed on one node. So the service-node mapping problem is modeled to be a minimum k -cut problem, where the edges that need to be cut represent the communication between microservices in different nodes.

5.2 Solving Microservice Mapping Problem Using Approximate Algorithm

Fig. 8 shows different cases in microservices (sequential and non-sequential). Mapping original microservice topology like

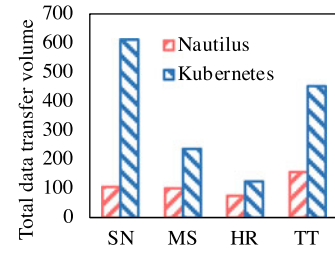


Fig. 9. Overhead comparisons between Nautilus and Kubernetes.

Fig. 8a on multi-node can be solved by some simple methods, such as Greedy Strategy (GS) and Dynamic Programming (DP). Unfortunately, these methods cannot be used in the structure like Figs. 8b and 8c, since GS and DP cannot find the optimal solution in polynomial time when they solve the complicated graph problems.

As the mapping problem can be transferred into a minimum k -cut problem, some graph algorithms can be used to solve this problem. In order to minimize the data transmission in the public network, it is necessary to minimize the sum of the cut edge weights so that the data corresponding to the edges with larger weights can be transferred to the global memory. However, it is impossible to get the optimal solution in polynomial time when $k \geq 3$.

To reduce the overhead of solving the minimum k -cut problem, Nautilus uses approximate algorithms to map microservices to appropriate nodes, which can quickly return results even in complex topological structures. Specifically, assuming that there are n microservices that need to be deployed on k nodes ($n \geq k$), the mapper will perform $k - 1$ iteration. In each iteration, the mapper will select a subgraph and divide it into two connected components. The algorithm calculates the global minimum cut for each subgraph and selects the lightest cut λ . In the process of calculating the global minimum cut, we choose the Ford-Fulkerson algorithm, in which the approximation ratio is $2 - 2/k$. Nautilus can reduce the data interaction between microservices from 39.2% to 82.4% compared with the Kubernetes deployment scheme as shown in Fig. 9. Nautilus does not use heuristic algorithms, such as simulated annealing or neural networks, as heuristic algorithms cannot guarantee the gap between the final result and the optimal solution.

5.3 Overhead of the Microservice Mapper

In order to evaluate the runtime overhead of the approximate algorithm. We use randomly generated graphs to simulate the process of the mapping problem. As shown in Fig. 10, the mapping strategy can quickly find the optimal task allocation plan. As the total number of microservices and the number of nodes increase, the solution time of the approximate algorithm will gradually increase.

In a combination of 100 microservices and 10 nodes, the solution time of the mapper does not exceed 400 ms.

Since the algorithm needs to calculate the minimum cut between every two vertices in one iteration. Therefore, if parallel threads are used to calculate the minimum cut of each vertices pair, the microservice mapping time will be further reduced. Our mapper is implemented in python's multiprocessing parallel library [51]. Fig. 11 shows the overhead of using parallel algorithms in a combination of 100

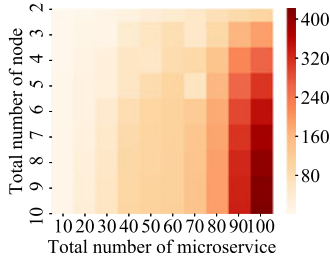


Fig. 10. The mapper's overhead on randomly generated services.

microservices and 10 nodes, where the x -axis represents the consumption of the number of processes. Observing the figure, when the number of processes is small (less than 16), the execution time of the mapper decreases significantly with the increase of the number of processes, which shows the effectiveness of the parallel algorithm. When the number of processes exceeds 16, the mapper execution time will slowly increase due to the synchronization overhead between processes.

6 CONTENTION-AWARE MICROSERVICE RESOURCE MANAGER

In this section, we present the contention-aware resource manager. Specially, we use deep reinforcement learning to allocate shared computational resources to microservices, aiming at minimizing the resource usage in low load and extending the peak load without QoS violation in high load.

6.1 Shared Resource Allocation Problem

Once the microservice mapping decision is completed, we need to allocate resources for the microservices based on the time-varying loads and the resource contention of co-located microservices, and the computational resources capacity of each node. The resource allocation problem can be described as a multi-objective optimization problem, as shown in Equation (1), where the objective function is to minimize the computational and network resource usage with the following constraints. First, the accumulated global memory capacity required by all microservices on a node shall not exceed the available capacity. Second, the total computational resource quotas allocated to microservice on one node should not exceed the available computational cores. Third, the total running time for the microservice application should be smaller than the QoS target. Forth, the supported throughput of the microservice application should be large than the users' request load. Table 2 lists the variables used in Equation (1).

Since the contradictory requirements, it is impossible to minimize both computational and network resource usages.

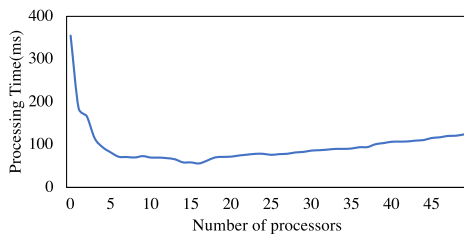


Fig. 11. The overhead of the mapper using parallel modules.

TABLE 2
Variables Used in the Optimization Problem

Variable	Variable Description
c_i	The cpu quota of the i th microservice
m_i	The memory quota of the i th microservice
$throughput$	The specified throughput of microservice
C_j	The total number of j th node's cpu core
M_j	The total capacity of j th node's memory
N_i	The number of instances of i th microservice
b_i	Network bandwidth of the i th node
D_i	Data transfer volume of the i th node
p_j	The set of co-located microservices in node j
$f(C, M)$	Predicted latency of the entire service
$tr(C, M)$	Predicted throughput of the entire service
Res	The total price of entire service
α, β, γ	Weighted factors between computation, memory capacity and network bandwidth

The first objective function affects the computing delay of microservices while the second affects the communication delay. There are multiple computing-communication delay configurations to meet the target QoS requirements. Considering they all have an impact on the end-to-end latency, there is a Pareto set satisfying the multi-objective resources allocation problem.

$$\begin{aligned}
 &\text{minimize} && \alpha \frac{\sum_{0 < i \leq n} c_i * N_i}{C} + \beta \frac{\sum_{0 < i \leq n} m_i * N_i}{M} \\
 &\text{minimize} && \sum_{0 < i \leq k} b_i \\
 &\text{s.t.} && \begin{cases} \sum_{i \in p_j} c_i N_i \leq C_j & j = 1, 2, \dots, k \\ \sum_{i \in p_j} m_i N_i \leq M_j & j = 1, 2, \dots, k \\ f(C, M) + \sum_{0 < i \leq k} D_i / b_i \leq QoS \\ \min tr(C, M) \geq throughput \end{cases}
 \end{aligned} \tag{1}$$

Assuming that a microservice-based application with n stages needs to be deployed in a cloud-edge continuum containing k nodes in different geographic locations, Nautilus obtains the optimal CPU usage, memory quota, and network bandwidth of each microservice by solving Equation (1).

6.2 Design of Reinforcement Learning Based Resource Management Agent

Solving the above optimization model relies on the prerequisites that the microservice performance characteristics are preknown, which is not practical at runtime. It is a non-trivial task to predict the end-to-end latency $f(C, M)$ and the throughput $tr(C, M)$ for given cores usage C and memory usage M . Prior work tends to predict the characteristics of each microservice by collecting and utilizing prior profiling information [12], [13], [52]. These methods require thousands of samplings to obtain enough data to estimate microservice characteristics. At the same time, the deployment strategy based on static optimal offline profiling is difficult to consider the contention of shared resources at runtime. Moreover, as the total number of microservices and nodes increases, the information that needs to be sampled will increase explosively. Instead, Nautilus chooses an RL-based model to solve the optimization problem without offline profiling. Meanwhile, the online decision-making deployment model can

capture the contention of shared resources at runtime, thereby eliminating the performance degradation of microservices.

Nautilus translates the resources management problem into a Markov decision problem (MDP) then solves the deployment of the resources by a Dueling DQN model[53]. Specifically, at time interval t_k , the learning agent observes s_k as represented by the resource usage of all microservices, including the number of CPU cores, memory capacity, and network bandwidth, while initializing action set a_k . Next, the agent uses a deep neural network to predict the end-to-end latency and peak throughput of microservices with different actions, where each action will lead to a new resource partition. After selecting the action with the largest performance, the agent will then get the corresponding reward R_k . Finally, the agent stores the state set s_k , action set a_k , and reward R_k in the experience pool as the training set for future reward evaluation.

Reward Function. The reward function refers to both the end-to-end latency and the throughput, which are collected by the load monitor at each step. According to the problem as stated in Equation (1), we define the reward as Equation (2). Intuitively, the reward function is heuristically designed encouraging the agent to choose resource partitions that just meet QoS and throughput requirements for minimizing resource usage. If the current resource allocation fails, the agent will find an acceptable solution closer to the partition that meets the requirements.

$$R = \begin{cases} Res & \text{otherwise.} \\ -\theta_1 \frac{QoS_r}{tr_r} + \theta_2 Res & QoS_r > 1 \quad \text{or} \quad tr_r < 1, \end{cases} \quad (2)$$

If the microservice application guarantees the QoS and throughput targets at runtime, the agent will get a *Resource Reward*. The Resource Reward (Res) is defined as the ratio of the capacity of the total resources to the measured resources consumption as shown in Equation (3). A larger Res indicates that microservices consume fewer resources while a higher value is added for saving resources.

$$Res = \alpha * \frac{CPU_m}{CPU_u} + \beta * \frac{Mem_m}{Mem_u} + \gamma * \frac{IO_m}{IO_u}. \quad (3)$$

On the other hand, when the QoS and throughput are not met, the agent will receive a *QoS Reward*. Specifically, we define the reward as the ratio of QoS_r and tr_r , where QoS_r represents the ratio of the measured QoS to the target QoS and tr_r represents the ratio of the target throughput to the measured throughput. If QoS_r is greater than 1, then the microservice application suffers from QoS violations. Similarly, if tr_r is less than 1, the microservice application has not reached the throughput target. When QoS_r is greater than 1 or tr_r is less than 1, we penalize the agent and impose a larger negative value. Besides, in QoS rewards, we also add resource rewards to encourage the agent to find the minimum resource division when approaching QoS and throughput goals. The parameter θ_1 and θ_2 controls the balance between QoS and resource consumption. To obtain better QoS, θ_1 shall have a higher value than θ_2 to ensure that QoS is more dominant.

From the perspective of the infrastructure providers, the resource allocation decisions shall be made from the usage

concern. While, from the perspective of users, besides the QoS or throughput, they also concern on the monetary cost. It is always desirable to lower the operational expenditure while still guaranteeing the QoS and the throughput. In this case, Nautilus weights each resource based on the cloud service pricing model. Specifically, α , β and γ in Equation (3) respectively represent the CPU, memory, and network costs per hour.

7 IO-SENSITIVE SCHEDULER

In this section, we introduce the IO-sensitive microservice scheduler to guarantee the quality of service(QoS) requirements of the microservice application by migrating critical microservices at runtime.

It has been confirmed that when IO-sensitive microservices are disturbed by shared resource contention, the end-to-end latency of microservice applications will explode significantly. Even if the IO bandwidth isolation mechanism is adopted, the shared resource expropriation from disk cannot be completely eliminated. Therefore, when there is external IO pressure that leads to the load imbalance in the cloud-edge continuum, it would be unwise to stick to the original mapping. In this case, Nautilus adaptively migrates the interfering microservices from high-load nodes to other idle nodes to ensure load balance and avoid QoS violations.

However, as mentioned in Section 3, the random migration mechanism cannot effectively alleviate the QoS violation of microservice applications, because of the inevitable IO contention for microservices in the high-load nodes. So Nautilus needs to carefully identify the critical microservices for migration to eliminate interference caused by contention for IO shared resources. There are two issues to be considered: 1) Which microservice stages need to be migrated. 2) Whether the data-intensive microservice stages need to be migrated.

We periodically sample each microservice and the overall microservice application to obtain the distribution feature of the duration of each microservice. To identify the critical microservices, we calculate the distance between each microservice and the overall microservice application. If the distance is larger, the duration of the microservice will be more likely to influence the overall end-to-end latency.

There are many methods to measure the distance between two distributions. Nautilus measures the impact of microservices on overall latency based on the Bhattacharyya coefficient [54], which effectively measures the similarity of two distributions. We assume that $p_i(x)$ represents the probability distribution of the duration of microservice i within a period of time, and $p(x)$ represents the probability distribution of the duration of the corresponding overall microservice application. Equation (4) calculates the probability distribution law of duration where t_1 to t_n represents the duration of all requests responded by the microservice during the sampling. It is worth noting that the duration of some abnormal microservices requests under IO interference will be much longer than the average, and in extreme cases, it can even reach 100 times, which leads to an imbalance in the probability distribution. In order to eliminate the influence of outliers, we use the logarithmic time to calculate the distribution law.

$$p(x) = ecdf(\log(t_1, t_2, \dots, t_n)). \quad (4)$$

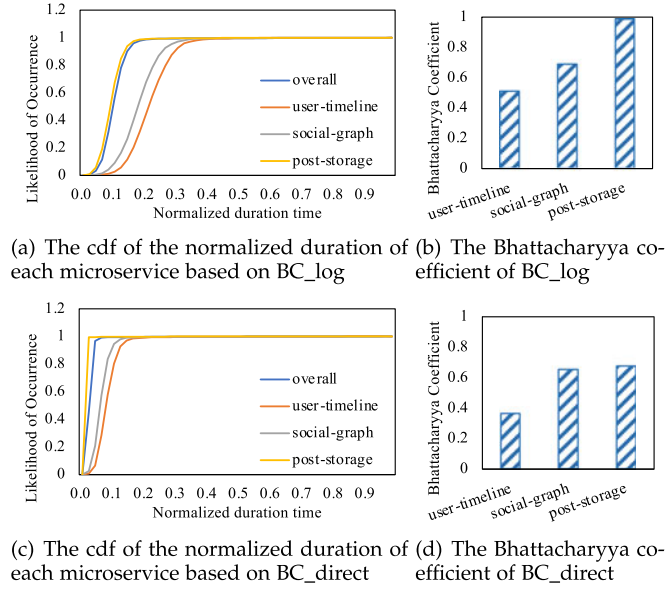


Fig. 12. The CDF of normalized duration and the Bhattacharyya coefficient of each microservice stages in SN under two strategies.

Equation (5) defines the BC (denoted as $BC(p_i, p_t)$) of the duration probability distribution of the microservice and the overall microservice application. If $p_i(x)$ and $p_t(x)$ are more similar, then $BC(p_i, p_t)$ is close to 1, and microservice i is more likely to cause end-to-end latency violations. On the contrary, if $p_i(x)$ and $p_t(x)$ are far apart, $BC(p_i, p_t)$ is at a lower value, and the time change of microservice i will not significantly affect the overall end-to-end latency.

$$BC(p_i, p_t) = \sum \sqrt{p_i(x)p_t(x)}. \quad (5)$$

According to this method, we carry out the assessment in the cloud-edge continuum. We verify the effectiveness of the two strategies, in which BC_log uses logarithmic duration to calculate the Bhattacharyya distance as discussed in Equation (4) and BC_direct uses the normal duration of microservices instead. Taking *socialNetworks*(SN) as an example. We consider all the three database-oriented microservices in SN since they frequently access disks and are disturbed by IO resources. We co-locate *post-storage* and the IO-sensitive task at the edge node and place the rest microservices in other nodes that are free from pressure interference. Figs. 12a and 12c show the CDF curve of database-oriented microservices in operation. Those microservices whose distribution closely fits the overall curve will be identified as critically affected microservices. As observed from Fig. 12a, *Post-storage* has become the critical microservice due to the impact of IO pressure, and its duration distribution is basically similar to the end-to-end latency time distribution.

Figs. 12b and 12d show the effectiveness of the BC_log and BC_direct. A higher bar indicates that the microservice is more likely to affect the overall end-to-end latency. Since the distribution of *post-storage* is similar to the total time, the BC distance is close to 1, which is much higher than the distance of other microservices. However, observed from Fig. 12d, BC_direct calculates the normal duration distance between microservices and cannot completely locate critical microservices. To conclude, using logarithmic time instead

of normal time is more effective for locating critically affected microservices.

If $BC(p_i, p_t)$ is greater than a certain threshold, it is considered that the duration of microservice i will affect the overall end-to-end latency and thus become a critical microservice. Then, Nautilus chooses to migrate this microservice to other idle nodes to avoid the expropriation of shared resources to ensure the quality of service of the microservice application.

There are also some methods to ensure the end-to-end latency of microservice applications by reducing the bandwidth occupied by IO load tasks or even stopping IO tasks [7], [11]. Nautilus chooses to migrate critical microservices to other nodes since Nautilus cannot confirm whether the co-located IO tasks have performance requirements, if we limit the resource occupation of these tasks, the performance of these tasks will be violated.

8 LOAD-AWARE SCHEDULER

In this section, we introduce a load-aware microservice scheduler to meet the quality of service (QoS) and throughput requirements of microservices in dynamic scenarios. It has been widely observed that the user requests exhibit the diurnal pattern [55]. In the consideration of user request dynamics, there is no one-fit-for-all solution. If we always stick to the original resource configuration provided by the mapper, the efficiency of the pipeline will be unbalanced, eventually leading to low throughput or even QoS violation. In particular, the edge nodes are comparatively with limited resource capacity, the microservices deployed on them have a tendency to become the bottleneck with the increased resource requirements.

To cater for the inevitable dynamic user requests, it is necessary to migrate microservices from busy nodes to idle nodes to reduce the resource contention of microservices. However, as mentioned in Section 3, there is a large amount of data communication between some microservice pairs, so it is still a challenging problem to carefully select microservices for migration to prevent a significant increase in microservice communication overhead.

Therefore, we design a load-aware migration strategy as illustrated in Fig. 13. When the performance monitor detects that a server in the cluster is working at full capacity, Nautilus calls the dynamic scheduler to migrate microservices from busy nodes to idle nodes. After the migration, the online resource manager will allocate more computational resources to the bottleneck microservices on this node to improve the efficiency of the pipeline. There are two problems in a migration operation, 1) which microservice shall be migrated and 2) which node it shall be migrated to.

To this end, we construct the communication increment model to measure the impact of migration on communication. Then we build a load-aware migration strategy based on the communication increment model. We assume that the microservice application is distributed deployed in a k -node cluster and microservice i is deployed at the node N_j . We define the communication increment caused by the migration of microservice i from node N_j to node N_k as shown in Equation (6).

$$\Delta C_{N_j \rightarrow N_k}^i = \sum_{m \in N_j} e_{im} - \sum_{m \in N_k} e_{im}. \quad (6)$$

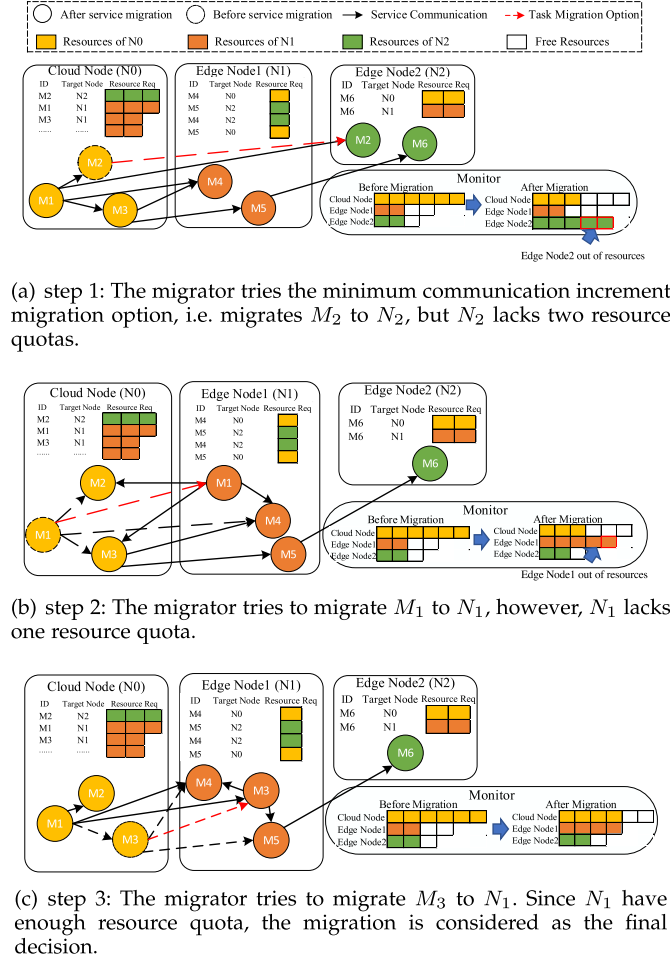


Fig. 13. Load-aware migration strategy.

In the Equation, the communication increment is defined as the subtraction of two components, where $\sum_{m \in N_j} e_{im}$ is the sum of the communication volume between the microservice deployed in the node N_j and the microservice i . When the microservice is migrated to N_k , the communication between the microservice on the node N_j and i will be transmitted through the public network, while the communication between the microservice and i on the node N_k will no longer consume the public network bandwidth.

In order to minimize the communication changes caused by the migration, Nautilus prefers to migrate the microservices with the smallest increase in communication. However, migration ensures the computational performance of microservices. If the target node does not have enough resources allocated to the migrated microservices, it will increase the duration of the microservice, thereby increasing the overall end-to-end latency. Based on this, we carefully select microservices and target nodes to avoid increased communication and performance degradation at the same time.

We assume that m_j microservices are deployed in node N_j . Since there are k nodes, there are a total of $m_j \times (k - 1)$ migration options. In order to determine the optimal option, we build a resource requirement table for each node. In each row of the table on node N_j , record the microservice m_i to be migrated, the target node N_k , the CPU core consumption cpu_i of the microservice, the memory capacity mem_i , and the communication increment $\Delta C_{N_j \rightarrow N_k}^i$. We sort the resource

requirement table in increasing increments of communication. In addition, Nautilus records the remaining quota of cpu core and memory capacity on each node. When the node becomes a congested node, we choose the option of the smallest communication increment satisfying the constraint that the two resources consumptions of the migrated microservice are less than the free resource quota in the target node, i.e., $cpu_i \leq cpu_{N_k}$ and $mem_i \leq mem_{N_k}$.

Fig. 13 illustrates the resource status, allocation process, and migration changes of the load-aware scheduler. When N_0 is full capacity, we select microservices on N_0 and migrate to other nodes based on the microservice topology and the resource utilization of each node. The monitor records the sum of microservice resource usage in node N_0 , N_1 , and N_2 and marks them as yellow, orange, and green respectively, while we mark the microservices in each node with the same node color. The upper half of each node records the microservice resource requirement table, including the target node of each migration option and the resource status of the microservice to be migrated, i.e., CPU resource quota. We only show the comparison of CPU core since the method of determining whether the memory capacity is sufficient is the same. Since the resource status in the migration option needs to be allocated by the target node, the microservice resource status in each resource option is marked with the color of the target node. Specifically, as shown in Fig. 13a the load balancer first tries to migrate M_2 to N_2 and allocate CPU cores on N_2 to M_2 since the migration option has the smallest communication increment. Unfortunately, since N_2 does not have enough CPU quota allocated to M_2 , i.e., $cpu_{M_2} > cpu_{N_2}$, the migrator rejects the migration option. Instead, Fig. 13c shows that the final migration change will be migrating M_3 to node N_1 as node N_1 has enough resources to guarantee the allocation process.

9 EVALUATION

In this section, we first evaluate the effectiveness of Nautilus in improving the resource utilization efficiency while satisfying the QoS requirement of microservices in the cloud-edge continuum. We also dive into Nautilus for inspecting the effectiveness of the IO-sensitive microservice scheduler and the load-aware microservice scheduler.

9.1 Experiment Setup and Compared Baselines

We perform all the experiments in Nautilus on the cloud-edge continuum with three nodes. Table 1 in Section 3 has already summarized the detailed experiment configuration and microservice applications used as benchmarks. Nautilus does not depend on specified software and hardware and can be easily set up on the later cloud or edge equipment.

In our experiment, the load of cloud services is defined as the number of requests that a microservice can handle per second. In the experiment, we use the load generator "wrk2"[56] to generate different microservices' loads for testing microservices' performance. Besides, we use Jaeger [57] to track the duration of each microservice and the overall end-to-end latency. In order to verify the effectiveness of the QoS guarantee of microservices with external IO pressure interference, we run a workload composed of *fio* in the cloud, which is the same as discussed in Section 3.

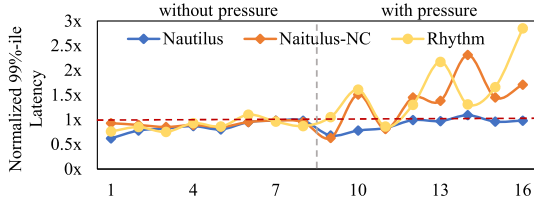


Fig. 14. The 99%-ile latency of the benchmarks (normalized to the QoS target). The left part of the figure shows the end-to-end latencies without IO pressure while the right part of the figure shows the latencies with external IO pressure.

We compare Nautilus with Kubernetes that adopts the best-effort deployment (discussed in Section 3), and Rhythm [11]. Rhythm maps microservices to distributed datacenters according to their functions. In addition, Rhythm sets a *loadlimit* for the entire service according to the resource configuration in the datacenter to ensure the stability of the end-to-end latency of the microservices. We continue to reduce the computational resource capacity of the cloud-edge continuum and record the minimum resource configuration that guarantees throughput requirements. In addition, we give Rhythm all the computation resources of the cloud-edge continuum and regard the *loadlimit* as the peak load level. When the performance of the microservice drops at runtime, Rhythm monitors the total IO bandwidth consumption of the microservices in the cloud node and allocates the remaining IO bandwidth to the co-location workload. However, it considers neither the mapping of microservices nor the load variation.

We first compare Nautilus with Rhythm in to guaranteeing the end-to-end latency of the overall microservice application in two scenarios: co-location with IO-intensive tasks and independent deployment. In the co-location scenario, microservice applications will suffer performance degradation due to interference from external IO pressure while in the independent scenario, the overall microservice application will be deployed in the cloud-edge continuum alone without external IO pressure. Second, we compare the difference in resource utilization efficiency between Nautilus and Rhythm in the case of guaranteeing the established goal of QoS. We define resource usage efficiency as the weighted average of the consumption of each resource (CPU core, memory capacity, and network bandwidth) in a microservice-based application, where the weight of each resource is determined by the cloud provider's pricing plan. Without loss of generality, we use AWS's t4g series pricing plan to measure resource utilization efficiency, where CPU core, memory capacity, and network bandwidth are 0.01054, 0.0053, and 0.0128 USD per hour, respectively.

9.2 Guaranteeing Required QoS Target

In this subsection, we evaluate Nautilus in guaranteeing the required end-to-end latency target in two scenarios. We deploy the benchmarks on the experimental cloud-edge continuum for 20 seconds, and collect the 99%-ile latency as well as the resource usage.

Fig. 14 shows the 99% tail latency of 16 scenarios without external pressure and with external pressure (4 benchmarks at low load and high load). We also compare Nautilus with Nautilus-NC, which does not include an IO-sensitive microservice scheduler. Nautilus-NC monitors the runtime tail latency and load pressure. When load imbalance occurs,

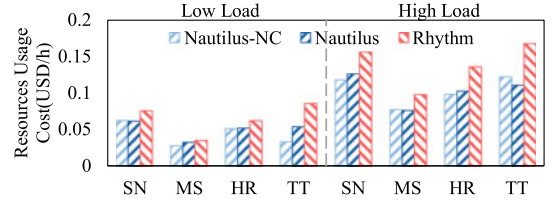


Fig. 15. The resources usage cost of microservice under external IO pressure on AWS.

Nautilus-NC selects the microservice with the smallest incremental change in communication for migration. As observed in Fig. 14, the QoS of benchmarks in most scenarios under no external pressure is satisfied with Rhythm, Nautilus, and Nautilus-NC. However, when microservice applications are suffered from external IO pressure, the tail latency of Nautilus-NC and Rhythm suffer from serious QoS violations.

Fig. 15 compares the resource usage of three runtime technologies under the co-location scenario. Compared to Rhythm, Nautilus reduces resource usage by an average of 20.0% while ensuring the QoS target. Meanwhile, although the resource consumption of Nautilus-NC is similar to that of Nautilus, Nautilus-NC cannot guarantee microservices less than the established requirements.

Rhythm suffers QoS violation due to lack of migration mechanism. When microservices and co-location tasks compete for IO shared resources, Rhythm allocates more computational resources to the disturbed nodes. However, the microservices on this node still suffer from external IO pressure, which leads to increased latency. On the other hand, Nautilus-NC pays more attention to communication changes during the migration and ignores the sensitivity of microservice to external IO which leads to the migration of non-critical microservices. Therefore, after the migration, the disturbed critical microservices still suffer from IO resources contention and the end-to-end latency violation has not improved. On the contrary, Nautilus effectively migrates critical microservices that are under external IO pressure, thereby eliminating the interference caused by external pressure on microservices.

9.3 Resources Utilization Efficiency

In this subsection, we evaluate the effectiveness of Nautilus in improving resource utilization efficiency while ensuring QoS requirements. Since Rhythm cannot guarantee the QoS targets under external IO pressure, we compare Nautilus with Rhythm without external IO pressure. We show the difference in resource utilization efficiency of Nautilus and Rhythm from two aspects. First of all, we compare the resource usage of the benchmarks at low load. Second, we show the supported peak loads of the benchmarks with Nautilus and Rhythm, when the hardware is fixed. The first comparison reveals the effectiveness of Nautilus in reducing resource usage, and the second comparison reveals its ability to better utilize all the resources.

Fig. 16a shows the resource usage of the benchmarks with low load and high load. In the figure, the line "network usage" shows the network resource usage of the benchmark with Nautilus normalized to its counterpart with Rhythm. As observed from the figure, Nautilus reduces the CPU usage and the network bandwidth usage of the benchmarks by 23.9% and 53.4% respectively compared with Rhythm.

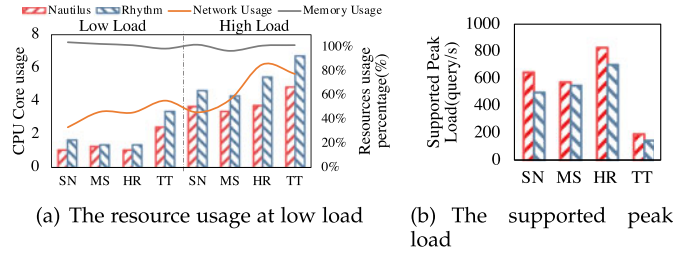


Fig. 16. The resource usage of the benchmarks with low loads, and the supported peak load of the benchmarks under the same computational resources with Rhythm and Nautilus.

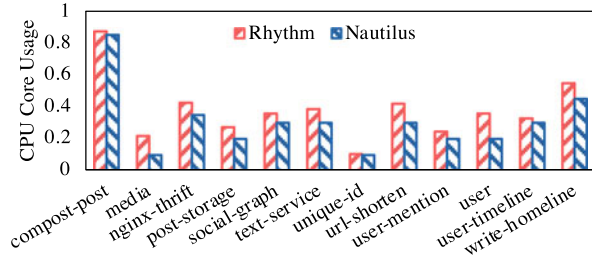


Fig. 17. CPU resource usage of the microservices in Rhythm and Nautilus.

Meanwhile, Fig. 16b shows the peak supported load of the benchmarks with Nautilus and Rhythm on our cloud-edge continuum. As observed, Nautilus improves the peak loads of the benchmarks by 20.6% on average compared with Rhythm. Nautilus improves the peak loads because it is able to support the same load with fewer computational resources. Nautilus is able to use the saved resources to process more users' requests.

To better understand the reasons that Nautilus reduces the resource usage, Fig. 17 shows the amount of the CPU resources allocated to each individual microservice of a benchmark SN at high load without external IO pressure. Nautilus reduces the resource usage of almost every microservice. This is mainly because Nautilus understands the resource contention between the microservices through reinforcement learning at runtime. In this case, Nautilus is able to identify the critical path of microservices and allocate fewer resources to other microservices.

To conclude, as Nautilus reduces the usage of CPU cores, memory space, and network bandwidth, it is able to run a microservice-based user-facing service at a lower economic cost. Fig. 18b shows the economic cost of the benchmarks with Nautilus and Rhythm. Nautilus reduces the deployment costs of the benchmarks by 29.9% on average compared with Rhythm.

9.4 Effectiveness of the Load-Aware Microservice Scheduler

The load-aware microservice scheduler monitors the QoS and the load of the entire application and migrates microservices accordingly. To verify the effectiveness of the dynamic scheduler, we compare Nautilus with Nautilus-NM, a variant of Nautilus that disables the dynamic scheduler. In this experiment, the peak load of each benchmark is configured to be its peak supported load with Nautilus.

Fig. 19a shows the 99%-ile latency of the benchmarks with Nautilus-NM. As observed from this figure, Nautilus-NM is not able to satisfy the QoS requirement of the benchmarks at

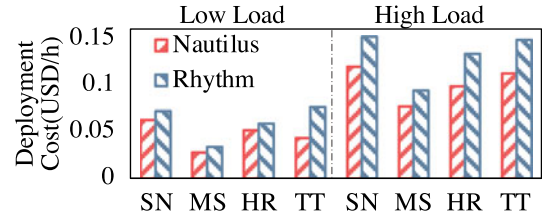
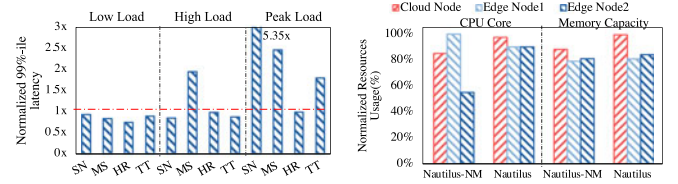


Fig. 18. The resources usage cost of deploying microservice without IO pressure on AWS.



(a) The 99%-ile latency of the benchmarks with Nautilus-NM (b) The resources usage of SN on each node under the peak load

Fig. 19. The 99%-ile latency of the benchmarks with Nautilus-NM and the resource usage of SN on each node under the peak load.

the high load, especially the peak load. For instance, the 99%-ile latency of the benchmark SN at the peak load is 5.35X of the QoS target with Nautilus-NM.

Nautilus-NM results in the QoS violation because it does not adjust the microservice mapping when the load changes. When the load of a user-facing service increase, if we stick to use the original mapping scheme, the entire service may suffer unbalanced shared computational resources partition. In this case, the microservice pipeline is inefficient and even leads to serious QoS violations. The dynamic scheduler detects busy nodes and migrates microservices from them to idle nodes.

As an example, Fig. 19b reports the CPU and memory usage resource usage of the benchmark SN at the peak load with Nautilus-NM. As observed, edge node resource usage exceeds 90%, while other nodes' shared computational resource usages remain low.

9.5 Adapting to the Load Variation

As stated before, the optimal microservice mapping and resource allocation change with the load of a user-facing application. Nautilus uses reinforcement learning to adapt to load changes and allocate computational resources to each microservice, which results in a high resource allocation overhead. For example, when the resource load of microservices increases, the resource requirements of each microservice increase, and the resource manager needs to retrain a larger resource configuration, which leads to QoS violations of the microservices at runtime. In this section, we evaluate Nautilus to adapt to load changes at runtime and guarantee QoS mechanisms at runtime.

We divide Nautilus into a training phase and a runtime phase. In the training phase, Nautilus pretrains microservice resource allocation strategies for several load levels. In the runtime phase, when the microservice load pressure increases, Nautilus first selects the minimum resource allocation strategy that exceeds the current pressure level. Second, Nautilus continuously reduces the microservice resource quota during runtime to avoid resource waste until the

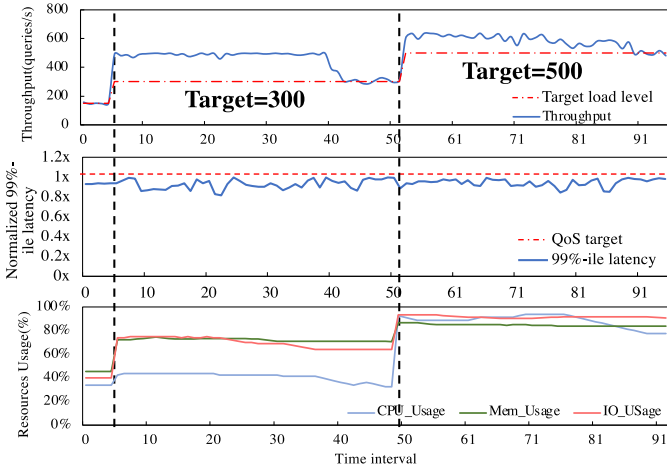


Fig. 20. Resources allocation on varying load.

supported load of the application is close to the current load level. Since the resource configuration of microservices always exceeds the resource requirements of microservices under the current load level, Nautilus can avoid QoS violations while reallocating computing resources at runtime.

Fig. 20 illustrates the resource allocation timeline of Nautilus when the load of a benchmark SN increases. In the upper part of Fig. 20, the blue line reveals the actual load in each time interval, and the red dotted line represents the target load. In the middle part, the red line implies the QoS target of the microservice application, while the blue line represents the end-to-end latency of the users' requests. In the lower part, "CPU_Usage", "Mem_Usage", and "IO_Usage" respectively represent the CPU core, memory, and network bandwidth consumption of the whole service.

In this experiment, we set 150 as the initial load pressure level of SN, and the user load increased to 300 and 500 respectively in the 5th and 52nd time intervals. In order to eliminate the QoS violation during the online RL re-training for the optimal resource allocation, we already pretrained three resource partition schemes under different load pressure levels, including 150, 500, and 600.

Fig. 20 shows the resource allocation strategy of Nautilus at runtime when the load pressure level increases. First, Nautilus chooses a higher load pressure solution to quickly ensure the QoS target of microservice applications. Specifically, Nautilus uses partitions under 500 and 600 load pressure levels in 5th and 52nd time intervals respectively. Second, because the current load pressure level is less than the resource partitions. Nautilus gradually reduces the resource usage of microservices so that the throughput stabilizes near the target load pressure. Since the throughput is always higher than the current load, the QoS target of the microservice is always guaranteed. Note that to ensure the QoS, Nautilus chooses excessive resource division to prevent QoS violations. Therefore, when the load pressure level rises to 500, Nautilus selects the corresponding resource partition when the load pressure level is 600 instead of 500.

10 DISCUSSION

In this section, we discuss the strategies in deploying microservice-based application with a heavy load in a larger scale

cloud-edge continuum. We also figure out the overhead of Nautilus.

10.1 Supporting Higher Application Load

The following analysis is based on two assumptions. First, the computing power of different edge nodes is similar. If k edge nodes are used, the computing power is k times that of a single edge node. In addition, as the computing power of edge nodes increases, the computing power of cloud nodes also increases proportionally. The computing power of an k -node cloud-edge continuum is k times that of a single-node cloud-edge continuum.

For a user-facing service that has n microservice stages, it is able to utilize at most n nodes without introducing more instances of the microservice. In this case, we deploy the n stages on different numbers of nodes ($2, 3, \dots, n$), and identify the peak load supported by $2, 3, \dots, n$ nodes with Nautilus (denoted by $PL_1, \dots, PL_n$). It is worth noting that the time required to identify the peak load of a service supported by n nodes is remaining short. This is because the decision scale of the reinforcement learning algorithm in Nautilus is only related to the number of microservice stages. Let PL denote the actual peak load of the service. If $PL \leq \max\{PL_1, \dots, PL_n\}$, Nautilus chooses to deploy the service on m nodes where m is the smallest value while PL_m is larger than PL .

On the other hand, if $PL > \max\{PL_1, \dots, PL_n\}$, we deploy multiple instances of the entire service, split and route the user requests to the multiple instances. In this scenario, the problem of minimizing resource usage can be expressed to be a target optimization problem with the goal of minimizing the use of computing nodes, either in the cloud or the edge. For the optimization problem, the constraint is that the peak load supported by multiple service instances should be larger than the actual peak load. Equation (7) describes the optimization problem. In the equation, t_i and p_i represent the supported peak load and resource usage of the microservice application in the i -node cloud-edge continuum, s_i represents the number of the i -node cloud-edge continuum used by the service.

$$\begin{aligned} &\text{minimize} && \sum_{i=2 \dots n} s_i p_i \\ &\text{s.t.} && \sum_{i=2 \dots n} s_i t_i \geq \text{throughput}. \end{aligned} \quad (7)$$

10.2 Overhead of Nautilus

Offline Overhead. Nautilus needs to analyze the topological structure of microservices offline and perform a small amount of sampling to obtain the amount of data exchange between microservices. This process takes less than 20s. Nautilus uses a low-latency approximate algorithm to ensure that the cost of allocating microservices is less than 10ms. But if parallel algorithms are used, Nautilus will be more efficient. *Resource allocation overhead.* As mentioned in Section 6, Nautilus uses the RL algorithm to solve optimization problems for microservices. Our results show that after 11000s of training, the agent can search for the optimal resource allocation. However, as discussed in Section 9.5, Nautilus adjusts the microservice resource configuration online and always guarantees the QoS target, so there is no need to offline the microservice application and re-train the agent. *Task migration overhead.* As described in Section 6, when Nautilus detects an unbalanced

resource configuration, Nautilus will migrate microservices in busy nodes to idle nodes. The cold start overhead of the container consumed by the migration is not more than 350 ms. *Runtime overhead.* First, the resource manager needs to predict the performance of the query by NN inference, which is completed within 10 ms. Furthermore, the task migration is also low-overhead. The complexity of the migration strategy is $O(mn)$, where n is the total number of nodes and m is the number of microservice stages on the node. Even if the number of nodes is larger than 100, the task migrator can still respond within 1 ms.

11 CONCLUSION

We propose Nautilus, a microservice deployment solution, and a runtime system on the cloud-edge continuum. Nautilus' task mapper places microservices that frequently interact with data on the same node to alleviate network communication overhead. We also propose the online resources manager and load-aware microservice scheduler to ensure that the microservices can use the minimum computational and network resources while ensuring the Quality of Service(QoS). Meanwhile, when a microservice application suffers from external IO pressure, an IO-sensitive microservice scheduler is designed to locate critical microservice and migrate them to idle nodes to guarantee the QoS requirement. Our experiments show that Nautilus can guarantee the required 99%-ile latency under external IO pressure while the state-of-the-art technique suffers from serious QoS violations. Besides, compared with traditional cloud datacenter resources management technology, Nautilus can reduce computational resources and network bandwidth usage by 23.9% and 53.4% respectively, while ensuring the required 99%-ile latency target. Nautilus focuses on minimizing the use of microservice resources while meeting throughput and QoS requirements at runtime. Although the training overhead of Nautilus will not affect the runtime throughput and QoS requirements of microservice applications, reducing the training overhead of the RL algorithm can speed up the searching process for the optimal resource configuration, which will be an interesting future work.

REFERENCES

- [1] Y. Gan *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2019, pp. 3–18.
- [2] Amazon Web service, 2021. [Online]. Available: <https://aws.amazon.com/>
- [3] Apple, 2021. [Online]. Available: <https://www.apple.com/>
- [4] Netflix, 2021. [Online]. Available: <https://www.netflix.com/>
- [5] J. Mars, L. Tang, R. Hundt, K. Skadron, and M. L. Soffa, "BubbleUp: Increasing utilization in modern warehouse scale computers via sensible co-locations," in *Proc. 44th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2011, pp. 248–259.
- [6] S. Chen, C. Delimitrou, and J. F. Martínez, "PARTIES: QoS-Aware resource partitioning for multiple interactive services," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2019, pp. 107–120.
- [7] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: Improving resource efficiency at scale," in *Proc. ACM/IEEE 42nd Annu. Int. Symp. Comput. Archit.*, 2015, pp. 450–462.
- [8] T. Patel and D. Tiwari, "CLITE: Efficient and QoS-Aware co-location of multiple latency-critical jobs for warehouse scale computers," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2020, pp. 193–206.
- [9] Edgex, 2021. [Online]. Available: <https://www.edgexfoundry.org/>
- [10] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [11] L. Zhao *et al.*, "Rhythm: Component-distinguishable workload deployment in datacenters," in *Proc. 15th Eur. Conf. Comput. Syst.*, 2020, pp. 1–17.
- [12] X. Hou, C. Li, J. Liu, L. Zhang, Y. Hu, and M. Guo, "ANT-Man: Towards agile power management in the microservice era," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, pp. 1098–1111.
- [13] Y. Gan *et al.*, "Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2019, pp. 19–33.
- [14] Production-grade container orchestration, 2020. [Online]. Available: <https://kubernetes.io/>
- [15] P. Pang *et al.*, "Sturgeon: Preference-aware co-location for improving utilization of power constrained computers," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 718–727.
- [16] N. Kulkarni, G. Gonzalez-Pumariaga, A. Khurana, C. A. Shoenmaker, C. Delimitrou, and D. H. Alboni, "CuttleSys: Data-driven resource management for interactive services on reconfigurable multicores," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchit.*, 2020, pp. 650–664.
- [17] Q. Chen *et al.*, "Alita: Comprehensive performance isolation through bias resource management for public clouds," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2019, pp. 1–13.
- [18] Q. Chen, Z. Wang, J. Leng, C. Li, W. Zheng, and M. Guo, "Avalon: Towards QoS awareness and improved utilization through multi-resource management in datacenters," in *Proc. ACM Int. Conf. Supercomputing*, 2019, pp. 272–283.
- [19] Q. Chen, H. Yang, J. Mars, and L. Tang, "Baymax: QoS awareness and increased utilization for non-preemptive accelerators in warehouse scale computers," in *Proc. 21st Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2016, pp. 681–696.
- [20] W. Zhang *et al.*, "Laius: Towards latency awareness and improved utilization of spatial multitasking accelerators in datacenters," in *Proc. ACM Int. Conf. Supercomputing*, 2019, pp. 58–68.
- [21] W. Zhang, Q. Chen, N. Zheng, W. Cui, K. Fu, and M. Guo, "Towards QoS-awareness and improved utilization of spatial multitasking GPUs," *IEEE Trans. Comput.*, to be published, doi: 10.1109/TC.2021.3064352.
- [22] P. Pang, Q. Chen, D. Zeng, and M. Guo, "Adaptive preference-aware co-location for improving resource utilization of power constrained datacenters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 441–456, Feb. 2021.
- [23] H. Zhao *et al.*, "CODA: Improving resource utilization by slimming and co-locating DNN and CPU jobs," in *Proc. IEEE 40th Int. Conf. Distrib. Comput. Syst.*, 2020, pp. 853–863.
- [24] R. Nishtala, V. Petrucci, P. Carpenter, and M. Själinder, "Twig: Multi-agent task management for colocated latency-critical cloud services," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2020, pp. 167–179.
- [25] H. Yang, Q. Chen, M. Riaz, Z. Luan, L. Tang, and J. Mars, "PowerChief: Intelligent power allocation for multi-stage applications to improve responsiveness on power constrained CMP," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 133–146.
- [26] K. J. Matteussi, C. F. R. Geyer, M. G. Xavier, and C. A. De Rose, "Understanding and minimizing disk contention effects for data-intensive processing in virtualized systems," in *Proc. Int. Conf. High Perform. Comput. Simul.*, 2018, pp. 901–908.
- [27] C. Delimitrou and C. Kozyrakis, "Quasar: Resource-efficient and QoS-aware cluster management," *ACM SIGPLAN Notices*, vol. 49, no. 4, pp. 127–144, 2014.
- [28] S. A. Javadi and A. Gandhi, "DIAL: Reducing tail latencies for cloud applications via dynamic interference-aware load balancing," in *Proc. IEEE Int. Conf. Auton. Comput.*, 2017, pp. 135–144.
- [29] W. Zhang *et al.*, "URSA: Precise capacity planning and fair scheduling based on low-level statistics for public clouds," in *Proc. 49th Int. Conf. Parallel Process.*, 2020, pp. 1–11.
- [30] S. Wang, Y. Guo, N. Zhang *et al.*, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.

- [31] M. Bunyakitanon, X. Vasilakos, R. Nejabati, and D. Simeonidou, "End-to-end performance-based autonomous VNF placement with adopted reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 2, pp. 534–547, Jun. 2020.
- [32] Y. Wang, Y. Li, T. Lan, and N. Choi, "A reinforcement learning approach for online service tree placement in edge computing," in *Proc. IEEE 27th Int. Conf. Netw. Protocols*, 2019, pp. 1–6.
- [33] Y. Li, D. Zeng, L. Gu, A. Zhu, and Q. Chen, "Task offloading in trusted execution environment empowered edge computing," in *Proc. IEEE 26th Int. Conf. Parallel Distrib. Syst.*, 2020, pp. 316–323.
- [34] X. Zhou *et al.*, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, Feb. 2021.
- [35] A. U. Gias, G. Casale, and M. Woodside, "ATOM: Model-driven autocalcing for microservices," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 1994–2004.
- [36] A. Kwan, J. Wong, H.-A. Jacobsen, and V. Muthusamy, "HyScale: Hybrid and network scaling of dockerized microservices in cloud data centres," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 80–90.
- [37] L. Suresh, P. Bodik, I. Menache, M. Canini, and F. Ciucu, "Distributed resource management across process boundaries," in *Proc. Symp. Cloud Comput.*, 2017, pp. 611–623.
- [38] H. Zhou *et al.*, "Overload control for scaling WeChat microservices," in *Proc. Symp. Cloud Comput.*, 2018, pp. 149–161.
- [39] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: ML-based and QoS-aware resource management for cloud microservices," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2021, pp. 167–181.
- [40] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ML-driven performance debugging in microservices," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2021, pp. 135–151.
- [41] A. Carrusca, M. C. Gomes, and J. Leitão, "Microservices management on cloud/edge environments," in *Proc. Int. Conf. Service-Oriented Comput.*, 2019, pp. 95–108.
- [42] I. Filip, F. Pop, C. Serbanescu, and C. Choi, "Microservices scheduling model over heterogeneous cloud-edge environments as support for IoT applications," *IEEE Internet of Things J.*, vol. 5, no. 4, pp. 2672–2681, Aug. 2018.
- [43] X. Hou *et al.*, "AlphaR: Learning-powered resource management for irregular, dynamic microservice graph," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2021, pp. 797–806.
- [44] Z. Li *et al.*, "Amoeba: QoS-Awareness and reduced resource usage of microservices with serverless computing," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 399–408.
- [45] N. Lazarev, S. Xiang, N. Adit, Z. Zhang, and C. Delimitrou, "Dagger: Efficient and fast RPCs in cloud microservices with near-memory reconfigurable NICs," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2021, pp. 36–51.
- [46] L. Bao, C. Wu, X. Bu, N. Ren, and M. Shen, "Performance modeling and workflow scheduling of microservice-based applications in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 9, pp. 2114–2129, Sep. 2019.
- [47] Aliyun, 2021. [Online]. Available: <https://www.aliyun.com/fio>
- [48] fio, 2020. [Online]. Available: <https://github.com/axboe/fio>
- [49] G. Kappes and S. V. Anastasiadis, "A user-level toolkit for storage I/O isolation on multitenant hosts," in *Proc. 11th ACM Symp. Cloud Comput.*, 2020, pp. 74–89.
- [50] R. S. Kannan, L. Subramanian, A. Raju, J. Ahn, J. Mars, and L. Tang, "GrandSLAM: Guaranteeing SLAs for jobs in microservices execution frameworks," in *Proc. 14th Eur. Conf. Comput. Syst.*, 2019, pp. 1–16.
- [51] Python multiprocessing library, 2021. [Online]. Available: <https://docs.python.org/dev/library/multiprocessing.html>
- [52] Q. Chen, H. Yang, M. Guo, R. S. Kannan, J. Mars, and L. Tang, "Prophet: Precise QoS prediction on non-preemptive accelerators to improve utilization in warehouse-scale computers," in *Proc. 22nd Int. Conf. Architectural Support Program. Lang. Oper. Syst.*, 2017, pp. 17–32.
- [53] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
- [54] T. Kailath, "The divergence and bhattacharyya distance measures in signal selection," *IEEE Trans. Commun. Technol.*, vol. CT-15, no. 1, pp. 52–60, Feb. 1967.

- [55] L. A. Barroso and U. Hölzle, "The datacenter as a computer: An introduction to the design of warehouse-scale machines," *Synthesis Lectures Comput. Archit.*, vol. 4, no. 1, pp. 1–108, 2009.
- [56] giltene, "wrk2." 2019. [Online]. Available: <http://github.com/giltene/wrk2>
- [57] Open source end-to-end distributed tracing, "Jaeger," 2020. [Online]. Available: <https://jaegertracing.io/>



Kaihua Fu (Student Member, IEEE) received the BSc degree from the University of Electronic Science and Technology of China, China. He is currently working toward the PhD degree under supervision of Dr. Minyi Guo in the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. His research interests include resource management in various architecture and high-performance computing.



Wei Zhang (Student Member, IEEE) received the BSc degree from the Huazhong University of Science and Technology, China. She is currently working toward the PhD degree under supervision of Dr. Minyi Guo in the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. Her research interests include high-performance computing and GPU resource management in datacenters.



Quan Chen (Member, IEEE) received the PhD degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China, in June 2014. He is currently a tenure-track associate professor in the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. His research interests include high-performance computing, task scheduling in various architectures, resource management in datacenter, runtime system and operating system.



Deze Zeng (Member, IEEE) received the BS degree from the School of Computer Science and Technology, Huazhong University of Science and Technology, China, in 2007, and the MS and PhD degrees in computer science from the University of Aizu, Aizu-Wakamatsu, Japan, in 2009 and 2013, respectively. He is currently a full professor in the School of Computer Science, China University of Geosciences, Wuhan, China. His research interests include edge computing, cloud computing, and future networking technologies.

He has authored 3 books and published more than 100 papers in refereed journals and conferences in these areas. He serves in editorial boards of the *Journal of Network and Computer Applications*, *Frontiers of Computer Science*, and *Open Journal of the Computer Society*. He is a senior member of CCF.



Minyi Guo (Fellow, IEEE) received the PhD degree in computer science from the University of Tsukuba, Japan. He is currently Zhiyuan chair professor, Shanghai Jiao Tong University, China. His research interests include parallel/distributed computing, compiler optimizations, embedded systems, big data and cloud computing. He is now on the editorial board of *IEEE Transactions on Parallel and Distributed Systems* and *Journal of Parallel and Distributed Computing*. He is CCF fellow.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.